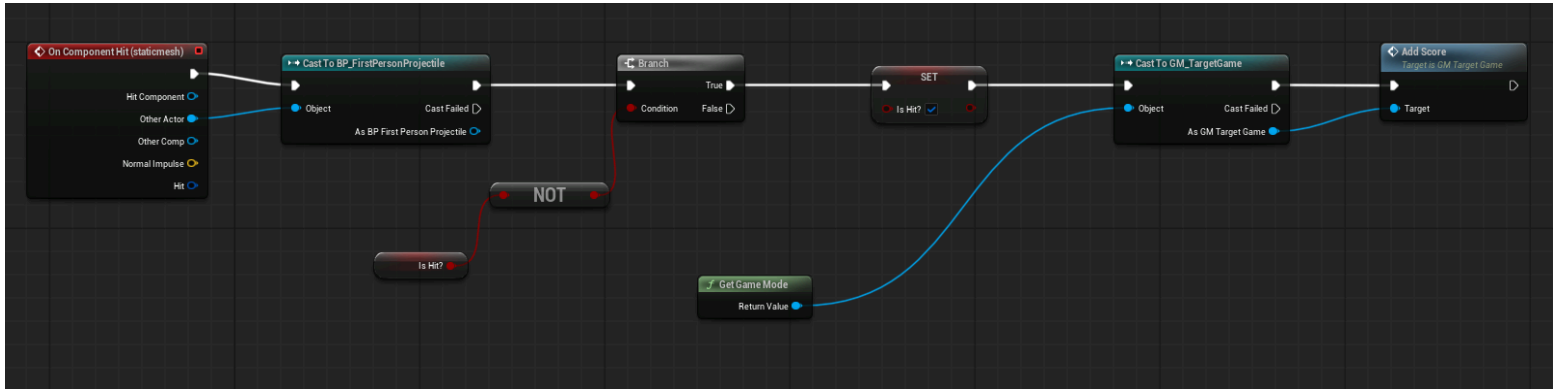


- BP_Target: The Blueprint for the target object



Function → When the target gets hit by the player's projectile, add score once, then mark the target as already hit

1. **OnComponentHit** → This starts the Blueprint

Definition:

- When the target's Static Mesh gets hit by something...
- **OtherActor** → The hit event gives information about the collision.

Definition:

- What actor hit me?

2. **Cast To BP_FirstPersonProjectile** → This node checks:
Was the thing that hit me a first-person projectile?

Definition:

- Only continue if the object that hit the target is the player's projectile. Meaning that if the target is hit by something else, nothing happens.

3. **isHit?** → Return True or False, Here it is used to remember whether the target has already been hit.

Definition:

- Returns **True** if it has already been hit and **False** if it has not been hit yet.

4. **NOT Boolean** → The Blueprint gets isHit?, then uses a NOT node.

Definition:

- If this target has not been hit before, then continue.

This flips the value:

- If isHit? is False → NOT makes it True
- If isHit? is True → NOT makes it False

5. **Branch : A Branch is like an if statement in programming.**

→ If the target has not been hit yet:

continue

Else:

do nothing

This prevents the player from getting unlimited points by hitting the same target again and again.

Definition:

- It checks the condition: NOT isHit?

6. **Set isHit? = True** → If the target has not been hit before, the Blueprint sets: isHit? = True

Definition:

- This target has now been hit. After this, if the projectile hits the same target again, the Branch will fail and no score will be added.

7. **Get Game Mode**

Then it uses:

- Get Game Mode

The Game Mode usually controls global game rules, like:

Score, Timer, Win condition, Player rules, Game state

So the target is asking: Where is the main game manager?

9. **Cast To GM_TargetGame** → This checks that the current Game Mode is the custom Game Mode:

GM_TargetGame

That is probably where your score system is stored.

So it says:

Get the current Game Mode and treat it as GM_TargetGame.

10. AddScore

Finally, it calls:

AddScore

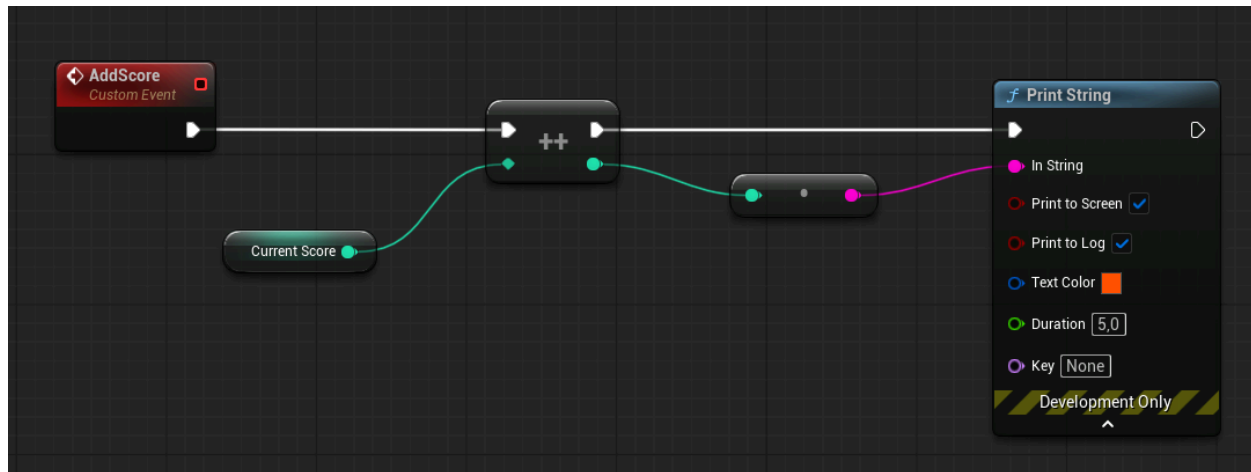
This means:

Increase the player's score.

So the full logic is:

- Target gets hit
- Check if it was hit by BP_FirstPersonProjectile
- Check if target was not already hit
- Mark target as hit
- Get the game mode
- Call AddScore
- The whole Blueprint in simple words
- When the target is hit by something:
 - If that thing is the player's projectile:
 - If this target has not been hit before:
 - Mark this target as hit
 - Add score

- GM_TargetGame: The Score system Blueprint inside the Game Mode.



1. AddScore Custom Event

This is the starting point.

Custom Event: AddScore

A **Custom Event** is a function created independently

So this means:

Whenever something calls **AddScore**, run this Blueprint logic.

In the previous target Blueprint, when the target gets hit by the projectile, it calls this AddScore event.

2. **currentScore** → This is an integer variable.

An integer means a whole number:

0, 1, 2, 3, 4...

So **currentScore** stores the player's current score.

At the start of the game, it is:

currentScore = 0

3. Increment Int

This node increases the score by 1.

So if:

currentScore = 0

after Increment Int, it becomes:

currentScore = 1

Then the next time:

currentScore = 2

Then:

currentScore = 3

So every time **AddScore** is called, the score goes up by 1.

4. **Conv_IntToString** → Convert Integer to String

5. **Print String** → This displays the score on the screen.

Whole flow:

AddScore event runs



Get currentScore



Increase currentScore by 1



Convert new score to text



Print score on screen

How it connects to your previous Blueprint

Your previous BP_Target Blueprint does this:

Target gets hit by projectile



Check if target was not hit before



Call AddScore

This GM_TargetGame Blueprint receives that call and does this:

AddScore



currentScore + 1



Print new score